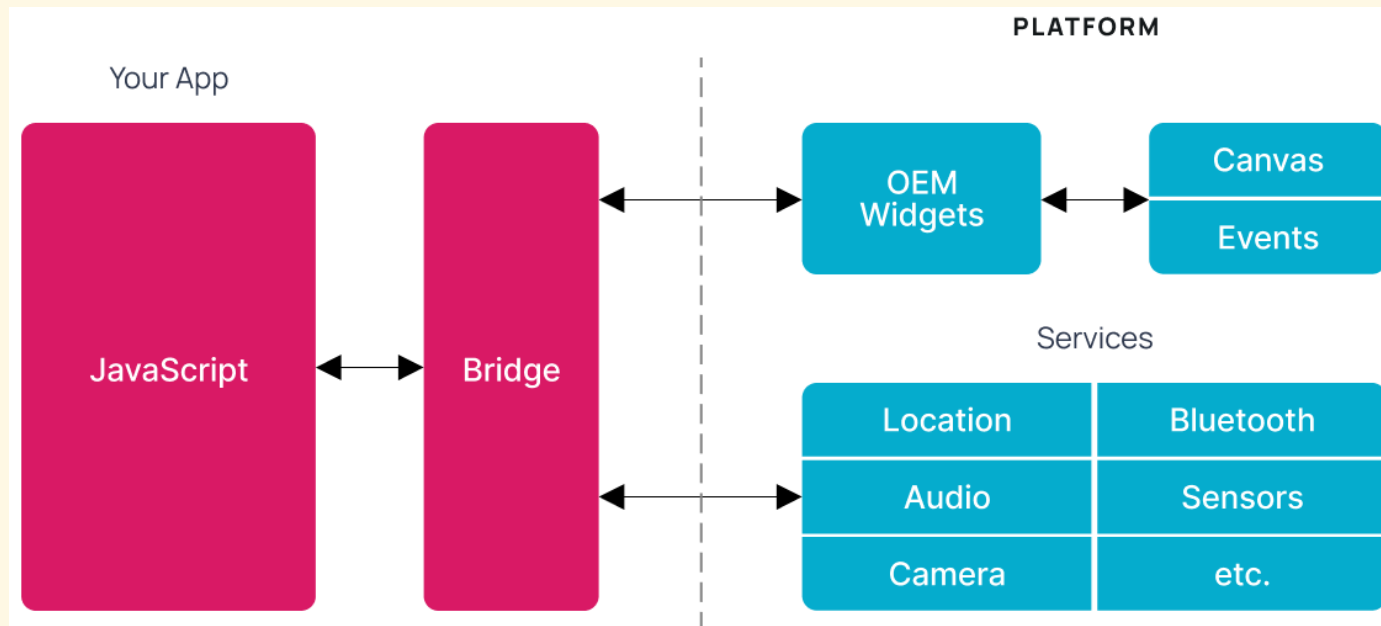

React Native



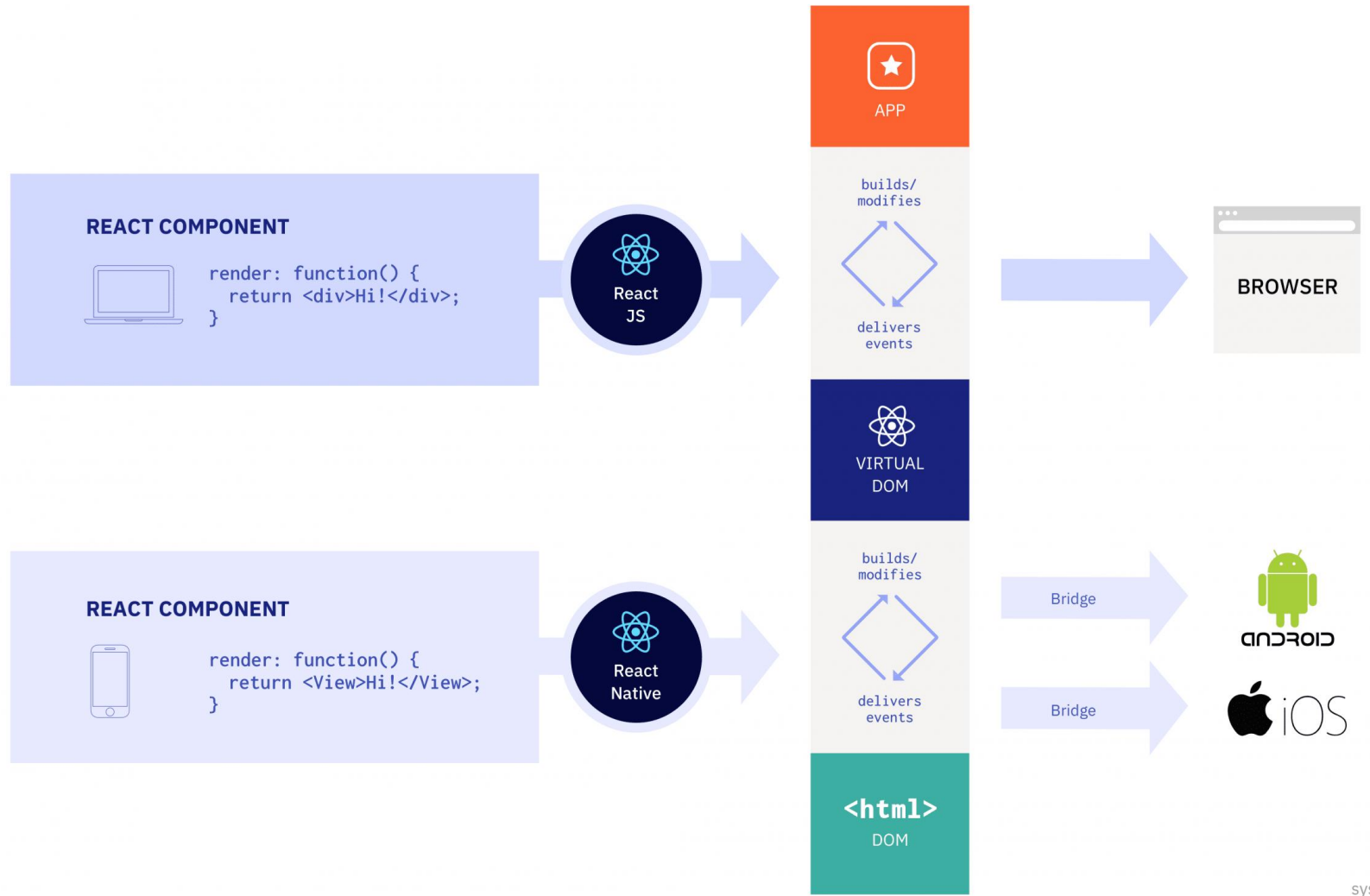
Overview

Introduction

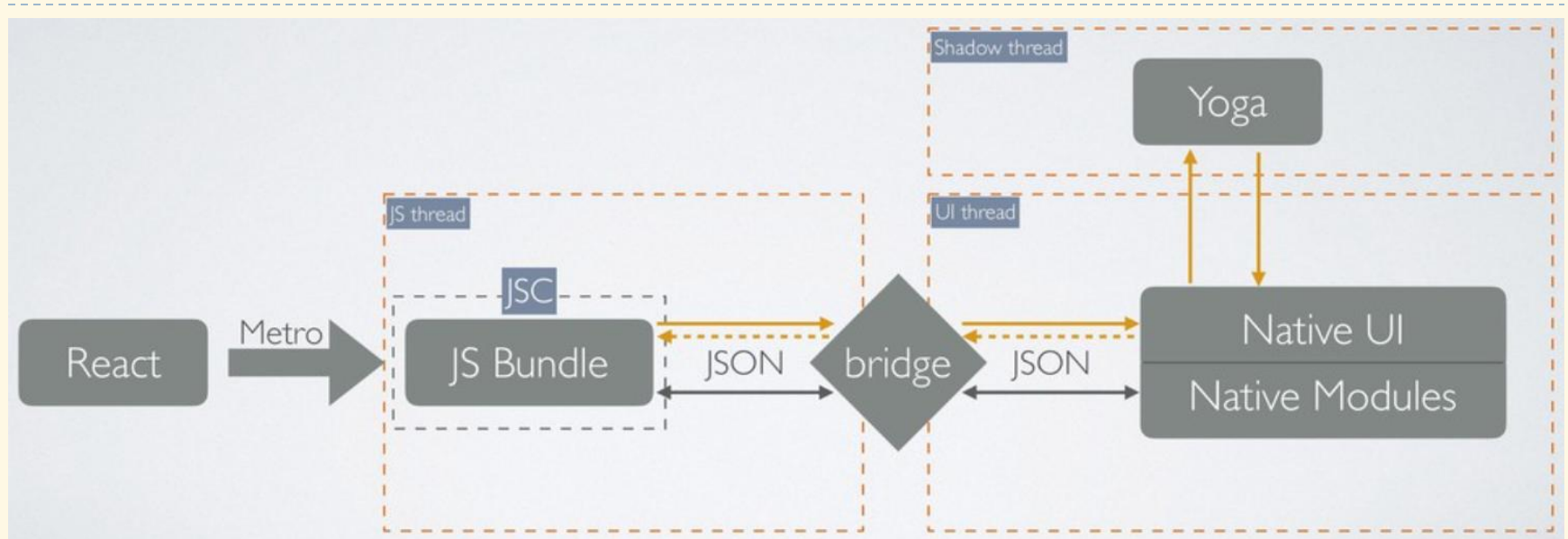
- ▶ React Native:
 - ▶ JavaScript framework for developing real, native apps for iOS and Android
 - ▶ Based on ReactJS – a framework mainly for web development
 - ▶ Using native components



From React to React Native

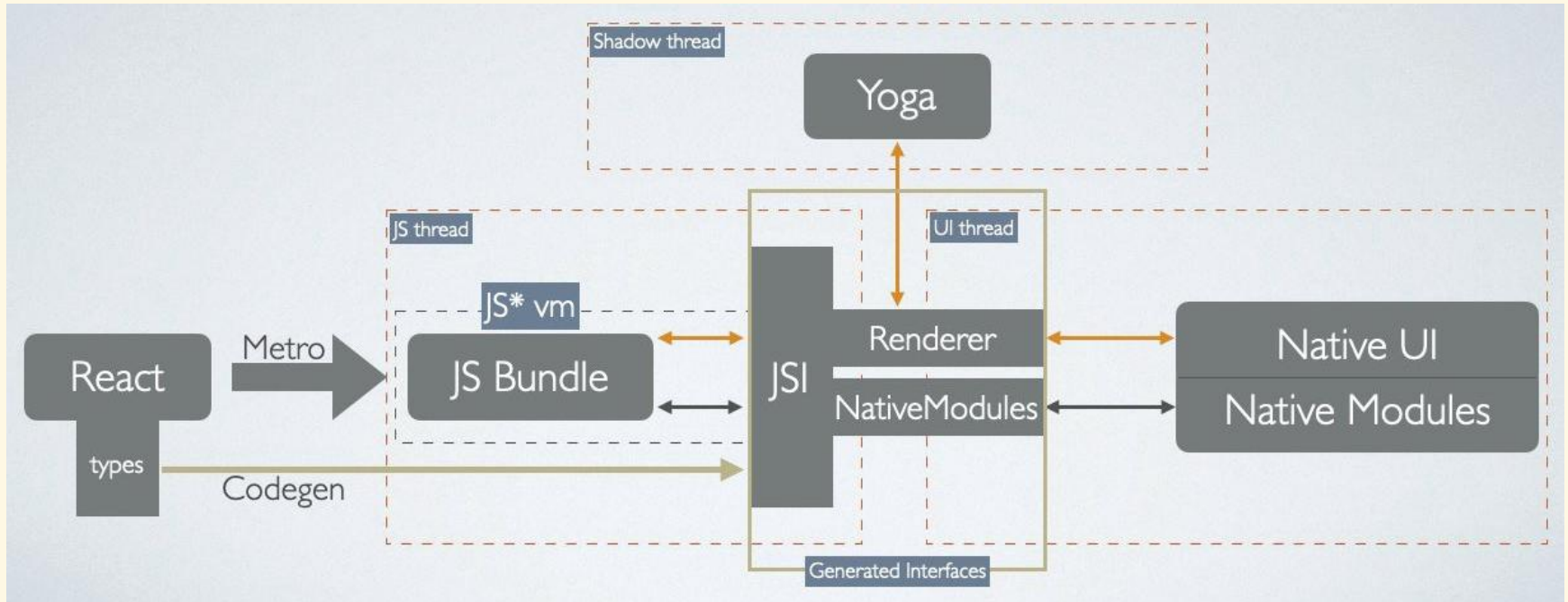


Old Architecture



- ▶ 3 separate threads
 - ▶ JavaScript thread: implementation of the app logics
 - ▶ UI thread (Java, Objective-C): native part working with the platform
 - ▶ Shadow thread (C++): calculation of UI layout
- ▶ Asynchronous communication through a Bridge

Current Architecture (Rolled-out in 2022)



- ▶ Bridge replaced by the JSI (JavaScript Interface): a direct interface between JS and C++
 - ▶ Shared ownership
 - ▶ Synchronous interoperability between threads

Choice of Development Framework

Feature	React Native CLI	Expo CLI
Easy project setup	—	+
Support of device's API: Bluetooth, Wi-Fi,...	+	—
Adding native modules in Java, Objective-C	+	—
Testing without Android Studio and XCode (No need a Mac for iOS development)	—	+
No build necessary to run the app	—	+
Building .apk and .ipa files	+	—
Debugging	—	+

- Get started with Expo, then switch to React Native CLI when being familiar

Environment Setup

- ▶ NodeJS
- ▶ Visual Studio Code
 - ▶ Install **React Native Tools** extension (for debugging)
- ▶ Expo CLI and React Native CLI
 - ▶ No need to install anything!
- ▶ Java SDK
 - ▶ <https://www.oracle.com/java/technologies/downloads/>
 - ▶ Choose latest LTS version
 - ▶ Set `JAVA_HOME` env variable to `<JavaSDK>` folder

Environment Setup (*cont'd*)

- ▶ Android Studio
 - ▶ <https://developer.android.com/studio>
 - ▶ Install Android Studio
 - ▶ Install Android SDK from Android Studio
 - ▶ Install Command-line Tools from Android Studio
 - ▶ Or using command line (**version number may need to be corrected**):
 - `sdkmanager "build-tools;30.0.2"`
 - ▶ Setup a virtual device
 - ▶ Set `ANDROID_HOME` env variable to `<AndroidSDK>` folder
 - ▶ Add to `PATH` env variable:
 - ▶ `<AndroidSDK>\platform-tools`
 - ▶ `<AndroidSDK>\cmdline-tools\latest\bin`
- ▶ Install Expo Go on the virtual or physical device:
 - ▶ <https://expo.dev/tools>

Create Project & Run Application

▶ React Native CLI

- ▶ `npx @react-native-community/cli init`
- ▶ `npm start`
- ▶ `npm run android`

▶ Expo CLI

- ▶ `npx create-expo-app <project-name>`
- ▶ `npm run web`
- ▶ `npm run android`
- ▶ `npm start`

Basics

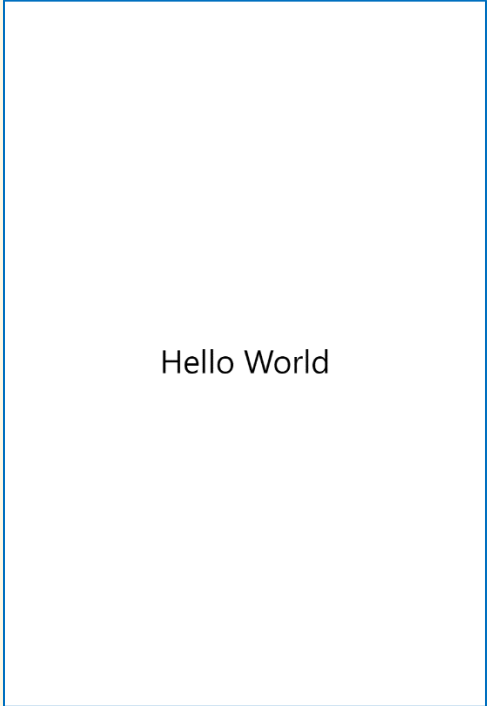


Hello World

```
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={{fontSize: 30}}>Hello World</Text>
    </View>
  );
}

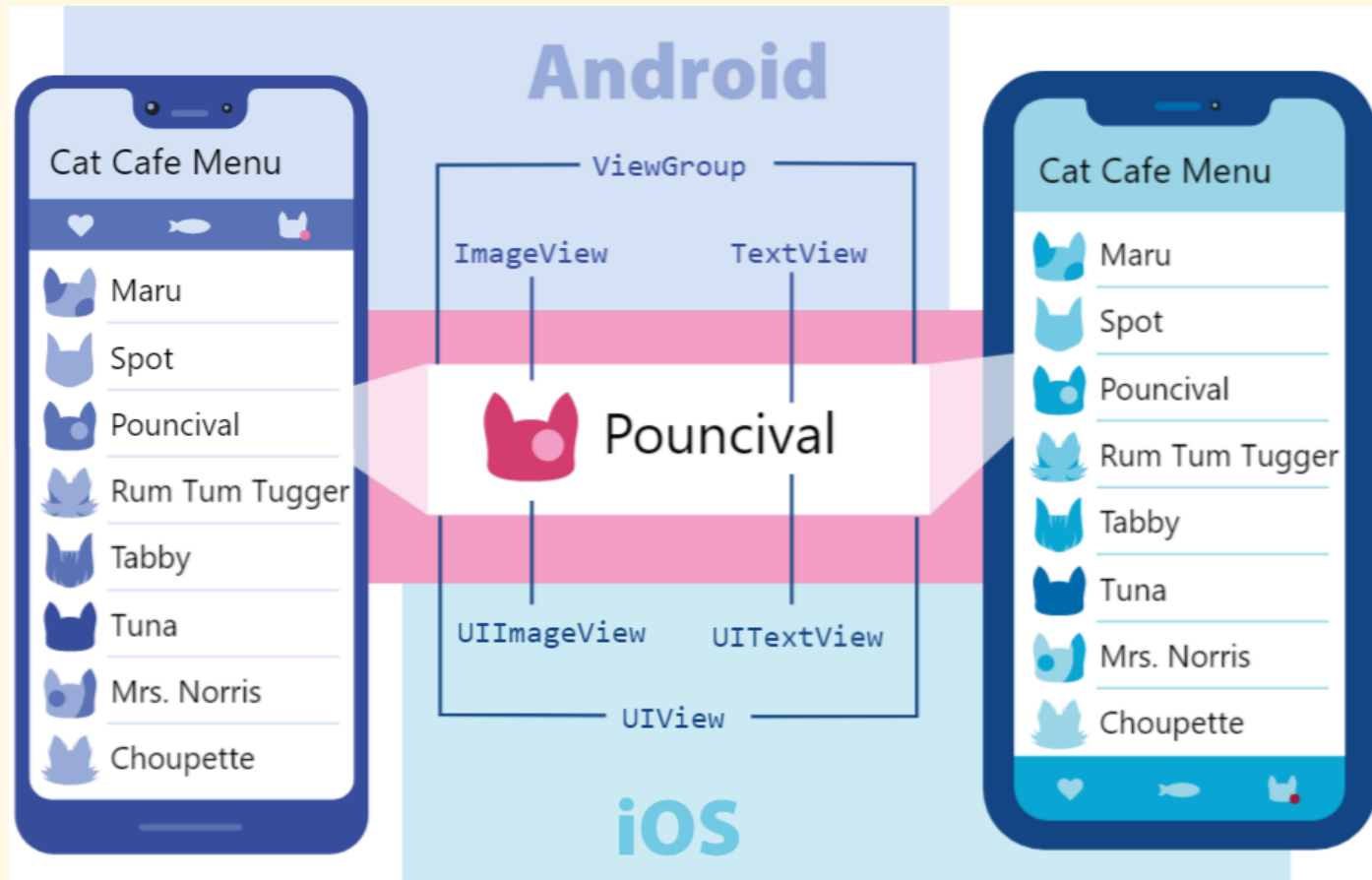
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```



Hello World

View

- ▶ A basic building block of UI



Native Components

- ▶ Components that are backed by the platform
- ▶ Most used:

React Native	Android	iOS	HTML
<code><View></code>	<code><ViewGroup></code>	<code><UIView></code>	<code><div></code>
<code><Text></code>	<code><TextView></code>	<code><UITextView></code>	<code><p></code>
<code><Image></code>	<code><ImageView></code>	<code><UIImageView></code>	<code></code>
<code><ScrollView></code>	<code><ScrollView></code>	<code><UIScrollView></code>	<code><div></code>
<code><TextInput></code>	<code><EditText></code>	<code><UITextField></code>	<code><input type="text"></code>
<code><Button></code>	<code><Button></code>	<code><UIButton></code>	<code><button></code>

Exercises

1. Rewrite the “hello world” app using class component
2. Play fun with simple components and styling: `<Text>`,
`<View>`

<Text>

- ▶ Represents text nodes
- ▶ Style inheritance possible in nested text elements

- ▶ Example:

```
<Text style={{ fontWeight: 'bold' }}>  
  I am bold  
  <Text style={{ color: 'red' }}>  
    and red  
  </Text>  
</Text>
```


<Button>

```
<Button title="Click me" onPress={...} />
```

- ▶ **Props:** title, color, disabled
- ▶ **Events:** onPress

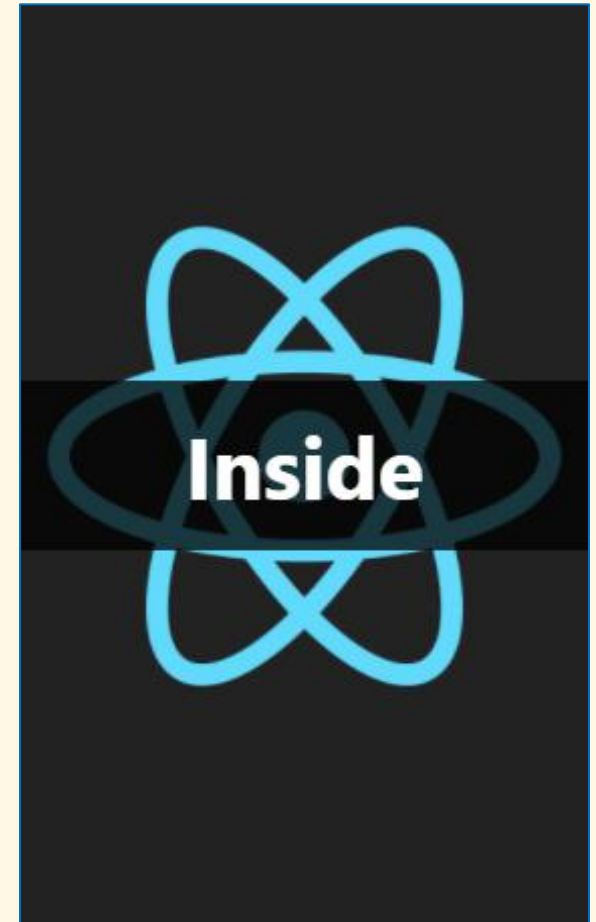
<Image>

- ▶ `<Image style={styles.logo} source={require("path/logo.png")} />`
- ▶ `<Image style={styles.icon} source={{uri: "data:image/png;base64,iVBORw0KGgo..."}} />`
- ▶ `<Image style={styles.icon} source={{uri: "https://..."}} />`
- ▶ **Props:** source, resizeMode (cover, contain, stretch, repeat, center)
- ▶ **Events:** onError, onLoad

<ImageBackground>

- ▶ Like <Image> but includes children

```
<ImageBackground
  source={...}
  style={...}
>
  <Text>Inside</Text>
</ImageBackground>
```



<TextInput>

```
function App() {  
  const [name, setName] = useState("");  
  return <View style={styles.container}>  
    <TextInput style={styles.input}  
      placeholder="Enter a name"  
      onChangeText={val => setName(val)}  
      defaultValue={name} />  
    <Text style={styles.text}>  
      Name: {name}  
    </Text>  
  </View>  
}
```

John Doe
Name: John Doe

- ▶ Try the example without `onChangeText` and see what happens

<TextInput>

- ▶ **Props:** placeholder, defaultValue, textAlign, maxLength, multiline, autofocus, returnKeyType
- ▶ **Events:** onChangeText, onFocus, onBlur, onKeyPress

<ScrollView>

- ▶ **Scrollable view with bounded height**
- ▶ **Props:** `horizontal`,
`showsHorizontalScrollIndicator`,
`showsVerticalScrollIndicator`,
`persistentScrollbar`, `pagingEnabled`
- ▶ **Events:** `onScroll`

<FlatList>

- ▶ Performant interface for rendering basic, flat lists
- ▶ Renders only elements that are currently showing on the screen
- ▶ `<FlatList`
 `data={dataArray}`
 `renderItem={({item}) =>`
 `<Item title={item.title}/>}`
 `keyExtractor={item => item.id}`
 `/>`
- ▶ **Props:** `data`, `renderItem`, `keyExtractor`, `inverted`
- ▶ **Events:** `onEndReached`
- ▶ Other props & events inherited from `<ScrollView>`

keyExtractor Prop

- ▶ A function that extracts a unique key for a given item at a specified index:
 - ▶ `(item: object, index: number) => string`
- ▶ When omitted, the default function checks for `item.key` and `item.id` and considers either of the two as the unique key
- ▶ Used by React Native for the re-rendering optimization when the list data is changed

Example

```
const initData = [{ id: 1, title: 'Hanoi'}, { id: 2, title: 'Danang'},...];

function Item({info}) {
  console.log('Rendering: ', info);
  return <View style={styles.item}>
    <Text style={styles.itemInfo}>{info.title}</Text>
  </View>
}

// memo is used to prevent re-rendering when component props are not changed
const MemoItem = memo(Item, (prev, next) => prev.info.id == next.info.id);

export default function App() {
  const [data, setData] = useState(initData);
  return <>
    <Button onPress={() => setData(data.filter((e, i) => i !== 0))}
      title="Remove 1st item"/>
    <FlatList style={styles.container} data={data}
      keyExtractor={item => item.id}
      renderItem={({item}) => <MemoItem info={item} />}
    />
  </>
}
```

<SectionList>

- ▶ Performant interface for rendering sectioned lists

```
▶ const DATA = [{ title: "Main dishes",
  data: ["Pizza", "Burger", "Risotto"]
}, { title: "Sides",
  data: ["French Fries", "Onion Rings"]
}];
```

```
function App() (
  return <SectionList
    sections={DATA}
    keyExtractor={(item, idx) => item + idx}
    renderItem={({item}) => <Item info={item}/>}
    renderSectionHeader={({section}) =>
      <Header info={section} />}
  />
)
```

- ▶ Events and props are similar to <FlatList>

<Switch>

- ▶ **Boolean input**

- ▶ <Switch

```
trackColor=
  {{ false: "#767577", true: "#81b0ff" }}
thumbColor=
  {isEnabled ? "#f5dd4b" : "#f4f3f4"}
ios_backgroundColor="#3e3e3e"
onValueChange={toggleSwitch}
value={isEnabled}
/>
```

- ▶ **Props:** value, trackColor, thumbColor, disabled

- ▶ **Events:** onValueChange

<ActivityIndicator>

```
<View style={styles.indicator}>  
  <ActivityIndicator />  
  <ActivityIndicator size="large" />  
  <ActivityIndicator size="small"  
    color="#0000ff" />  
  <ActivityIndicator size="large"  
    color="#00ff00" />  
</View>
```

- ▶ **Props:** animating, color, hidesWhenStopped, size

Exercise

1. Render a list with different sorting options
2. Create a BMI calculator as in the figure
 - ▶ $BMI = w/h^2$ (w in kg, h in meters)
 - ▶ < 18.5 : underweight
 - ▶ $18.5 \div 25$: normal
 - ▶ $25 \div 30$: overweight
 - ▶ ≥ 30 : obese

The image shows a mobile application interface for a BMI calculator. It has a white background with a light blue header. The title 'BMI Calculator' is in bold blue text. Below the title, there are two input fields: 'Enter weight (kg):' with a value of 65 and 'Enter height (cm):' with a value of 165. Each input field has blue minus and plus buttons on either side. Below these, it shows 'Your BMI:' with the value 23.87 in green, and 'Status:' with the value 'Normal' in green.

BMI Calculator	
Enter weight (kg):	- 65 +
Enter height (cm):	- 165 +
Your BMI:	23.87
Status:	Normal

Touchable Wrappers

Introduction

- ▶ `<Button>` is limited in content presentation
- ▶ Use the following wrappers to response to press:
 - ▶ `<TouchableWithoutFeedback>`
 - ▶ `<TouchableHighlight>`
 - ▶ `<TouchableOpacity>`
 - ▶ `<TouchableNativeFeedback>`
 - ▶ `<Pressable>`

<TouchableWithoutFeedback>

- ▶ **No visual feedback to press**
- ▶ `<TouchableWithoutFeedback onPress={...}>`
 `<MyChildComponent />`
 `</TouchableWithoutFeedback>`
- ▶ **Props:** disabled
- ▶ **Events:** onPress, onPressIn, onPressOut, onLongPress, onFocus, onBlur

<TouchableHighlight>

- ▶ Darkens or lightens the background of the child element when pressed
- ▶ Inherits <TouchableWithoutFeedback>
- ▶

```
<TouchableHighlight
  activeOpacity={0.6}
  underlayColor="#DDDDDD"
  onPress={() => alert("Pressed!")}
>
  <MyChildComponent />
</TouchableHighlight>
```
- ▶ **Props:** activeOpacity, underlayColor
- ▶ **Events:** onHideUnderlay, onShowUnderlay

<TouchableOpacity>

- ▶ Decreases the opacity of the child element when pressed
- ▶ Inherits <TouchableWithoutFeedback>
- ▶

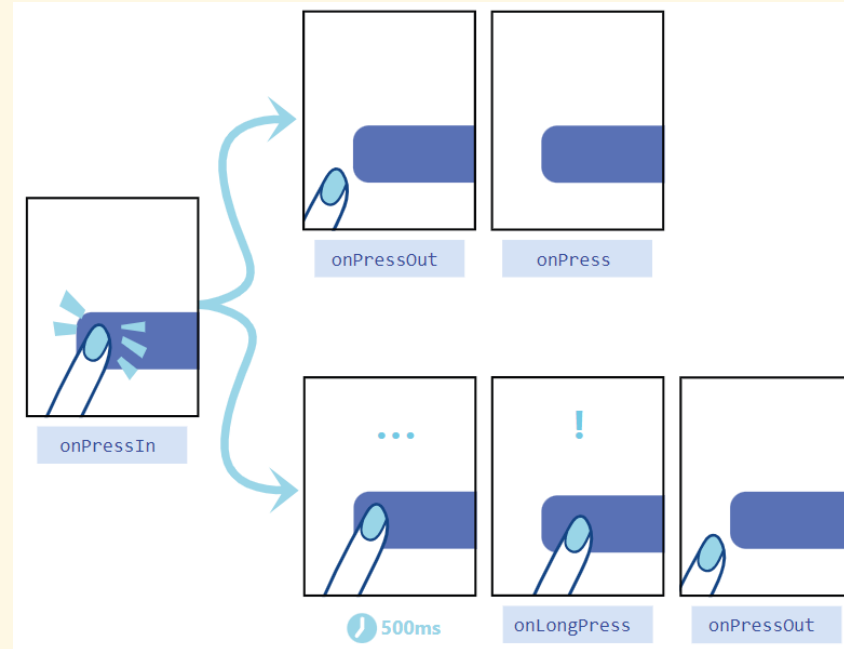
```
<TouchableOpacity
  activeOpacity={0.6}
  style={styles.button}
  onPress={onPress}
>
  <MyChildComponent />
</TouchableOpacity>
```
- ▶ Props: activeOpacity

<TouchableNativeFeedback>

- ▶ Uses native state drawable to display touch feedback (Android only)
- ▶ Inherits <TouchableWithoutFeedback>
- ▶ `<TouchableNativeFeedback background={TouchableNativeFeedback.Ripple("#ff0", false)}>`
 `<MyChildComponent>`
 `</TouchableNativeFeedback>`
- ▶ **Props:** background, useForeground

<Pressable>

- ▶ Detects various stages of press interactions on children
- ▶ No visual feedback
- ▶ `<Pressable onPress={...}>`
 `<Child1/>`
 `<Child2/>`
`</Pressable>`



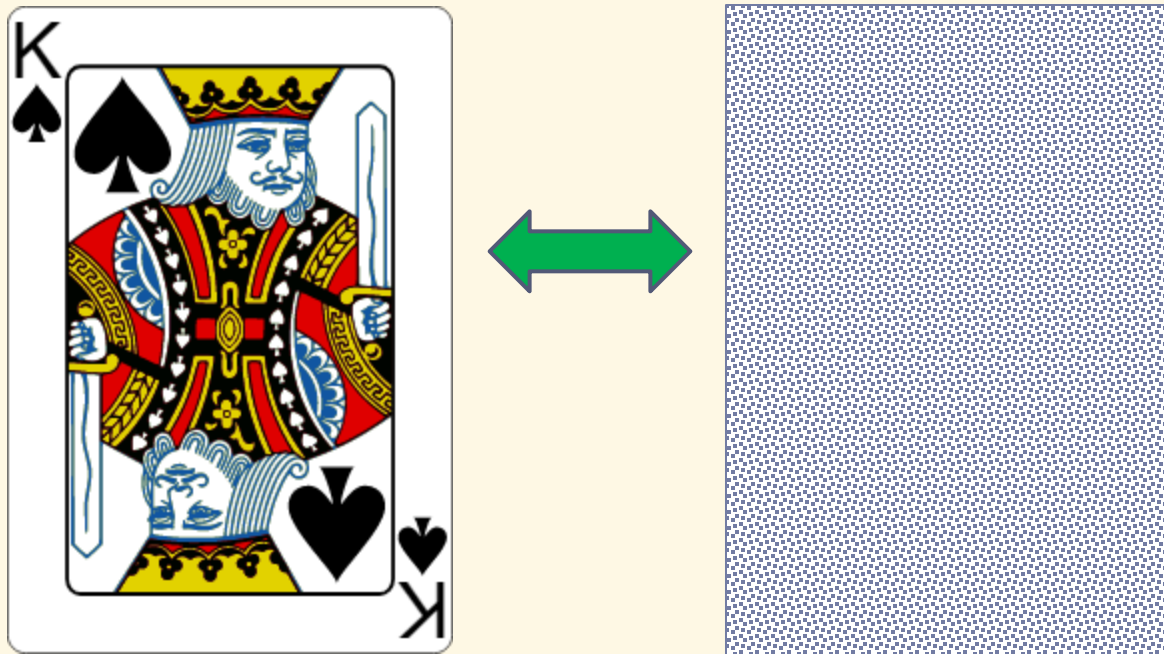
- ▶ **Props:** disabled
 - ▶ `children`, `style` props can be functions that receive a Boolean `pressed` property reflecting whether the component is currently pressed
- ▶ **Events:** `onPress`, `onPressIn`, `onPressOut`, `onLongPress`

<Pressable> with Visual Feedback

```
<Pressable style={({ pressed }) => [ {  
  backgroundColor: pressed ? "#7f7" : "#fff"  
}, styles.wrapper] }  
>  
  
  ({ pressed }) =>  
    <Text>  
      {pressed ? "Pressed!" : "Press Me"}  
    </Text>  
</Pressable>
```

Exercise

- ▶ Make a card that flips up and down when being pressed
 - ▶ Use `<Pressable>`



Homework

1. Write a memory card matching game
2. Create a simple calculator as in the figure

